

ECEN 631 - Robotic Vision

DirectDrive: Point-and-Drive with Stereo Vision

Team 1: Direct Drive

William Weigeshoff & Matthew Watler

Brigham Young University

April 2026

1. Introduction

For our final project we built a gesture-controlled RC car. You point at a spot on the ground, and the car drives there. It uses two cameras in a stereo setup to figure out where in 3D space you're pointing, then drives to that spot on its own.

We reused the Radxa RK3588 board and Traxxas RC car from our CyberTruck lane-following project. Both MIPI CSI cameras feed into MediaPipe for pose estimation, stereo triangulation gives us a 3D ground target, and the car steers toward it in a closed loop. Everything runs in real time on the Radxa.

2. System architecture

The code is split into modules that each do one thing. Frames come in from both cameras, go through pose estimation, then stereo triangulation figures out the 3D target. A state machine decides what the car does next. All the tunable numbers live in config.py.

Module	File	What it does
Pointing Detector	pointing_detector.py	MediaPipe pose + arm extension check
Stereo Vision	stereo_vision.py	Load calibration, undistort, triangulate
Ground Target	ground_target.py	Ray-plane intersection to find ground point
Navigator	navigator.py	State machine: WAITING / DRIVING / ARRIVED
Car Controller	car_controller.py	Serial commands to Arduino (steer, drive)
Config	config.py	All tunable parameters
Main Loop	main_pointing.py	Cameras, main loop, debug HUD

Table 1: Software modules.

2.1 Hardware

Component	Details
Compute	Radxa Rock 5T, RK3588 (aarch64, 8-core)
Left Camera	MIPI CSI @ 1280x720 (/dev/video31)
Right Camera	MIPI CSI @ 1280x720 (/dev/video22)
Stereo Baseline	~11.7 cm
Vehicle	Traxxas Splash 4x4 RC car
Motor Control	Arduino via USB serial (115200 baud)
Steering	-30 to +30 degrees
Pose Estimation	MediaPipe Pose (CPU)

Table 2: Hardware.

3. Pointing detection

We didn't train a custom gesture model. MediaPipe Pose already gives 33 body landmarks per frame, so we just check whether the arm is extended enough to count as pointing.

3.1 How it works

Each camera has its own MediaPipe instance. We compute an arm extension ratio (how straight the arm is, from 0 to 1) and if it is above 0.50, we call it a point. Both cameras have to agree on which arm is extended before we accept it. That requirement alone gets rid of most false positives.

3.2 Stereo correspondence by joint index

We didn't need feature matching between the two views. MediaPipe gives consistent joint indices (joint 12 is always the right shoulder, for example), so we triangulate by index rather than by visual similarity. This works well even when the person looks quite different from each camera's angle.

4. Stereo vision and ground target

4.1 Calibration

We calibrated the stereo pair at 1280x720 with a 10x7 checkerboard. That gave us intrinsic matrices, distortion coefficients, and the rotation/translation between cameras. The baseline is about 11.7 cm.

4.2 Triangulation

We take matching pixel coordinates from both cameras (shoulder, wrist, index fingertip), undistort them, and pass them to OpenCV's `triangulatePoints`. That gives us 3D positions in the left camera's coordinate frame.

4.3 Finding the ground target

With the shoulder and index fingertip in 3D, we form a ray from shoulder through fingertip and intersect it with the ground plane (170.5 mm below the camera). We originally used the wrist as the ray endpoint, but the fingertip turned out to be much better. The wrist only tells you where the arm is aimed; the fingertip tells you where the finger is actually pointing.

To keep things stable we run an EMA filter ($\alpha = 0.3$), reject outliers that jump more than 1 meter between frames, and require the ray to be aimed at least 5 degrees downward. The target also has to hold steady for 5 frames before the car will commit to driving. Without that, quick arm movements and noisy detections would trigger false starts.

5. Navigation

A state machine handles the drive cycle:

State	What happens	Next state
-------	--------------	------------

WAITING	Looking for a stable pointing gesture	Stable for 5 frames -> DRIVING
DRIVING	Steering toward the target	Close enough or timeout -> ARRIVED
ARRIVED	Stop the car	Immediately -> WAITING

Table 3: Navigation states.

While driving, the car re-estimates the target every frame. Steering is proportional ($\text{steer} = K_p * \text{heading_angle}$, clamped to ± 30 degrees) and speed is fixed at 75% throttle. A 10 second timeout stops the car if something goes wrong.

The car has no wheel encoders or IMU, so we can't dead reckon. Instead it just keeps looking at the target through the cameras. This works fine, but it means the person has to keep pointing while the car drives.

6. How we built it

We started with the stereo calibration (checkerboard at 1280x720) and verified triangulation by measuring objects at known distances. Then we built the pointing detector on a laptop with a single webcam using a `--webcam` flag that duplicates the frame as fake stereo. We tuned the extension threshold by testing with a few different people at different distances.

After that we wired up the stereo triangulation with the ray-plane intersection. We were using the wrist as the ray endpoint at first, but it was too inaccurate, so we switched to the index fingertip. That was a big improvement. We added EMA smoothing and outlier rejection on top to deal with noise.

For driving, we tested in `--no-drive` mode first to make sure state transitions fired correctly, then turned on motor commands and tuned speed, steering gain, and arrival distance on the actual car.

7. Results and video

The car detects pointing and drives toward the target. The visual servo keeps it roughly on course while moving. The person does need to stay visible and pointing while the car drives, since there is no odometry. If they move out of frame, the car holds its last heading until the timeout stops it.

Demo video: https://drive.google.com/file/d/1Vibrvtulc27LQvVlg_UyHWadJ3H6KQDe/view

8. Lessons learned

We tried scaling a lower-res calibration up to 1280x720 at first, and triangulation was way off. Recalibrating at the actual runtime resolution fixed it. Use the fingertip as the ray endpoint, not the wrist. The wrist only tells you arm direction, not where the finger is aimed. Also, camera height above the floor is surprisingly sensitive. A few centimeters off shifts the ground target by meters at longer distances. We had to measure it carefully and verify by pointing at known spots on the floor.